

FinanceBillingPlatform

Comprehensive Project Documentation

Ganesh Vardhineni

June 5, 2026

Contents

1	Executive Summary	2
2	Project Overview	3
3	Architecture	4
4	Technology Stack	6
5	Project Structure	7
6	Core Components & Implementation	8
7	API Endpoints	10
8	Database Schema	11
9	Authentication & Security	12
10	Configuration	13
11	Getting Started	14
12	Testing	17
13	Future Scope & Comparison	18

Chapter 1

Executive Summary

FinanceBillingPlatform is an enterprise-grade financial billing and invoice management system built with ASP.NET Core MVC, Web API, EF Core, and SQL Server. It enables organizations to create, manage, and track invoices, process payments, and maintain audit logs with JWT-based authentication and role-based authorization.

Key Features

- JWT Authentication with role-based authorization (Admin, FinanceUser, Client)
- Invoice Management with automatic status updates
- Payment Processing with revenue leakage prevention
- Audit Logging for compliance and accountability
- Clean Architecture with Repository Pattern
- EF Core with migrations and seed data
- Unit Testing with xUnit and Moq
- CI/CD pipeline with GitHub Actions
- Docker & Azure deployment strategy

Chapter 2

Project Overview

Purpose

The platform reduces revenue leakage by automating invoice/payment workflows and enforcing secure access.

Target Users

- Admin: Approves users, manages system oversight
- FinanceUser: Creates invoices, processes payments
- Client: Views invoices and payment history

Chapter 3

Architecture

Pattern

Clean Architecture + Repository Pattern

Layers:

- Presentation (MVC, Razor Pages, Controllers)
- Business Logic (Services, SOLID principles)
- Infrastructure (Repositories, DbContext)
- Data Access (EF Core, ADO.NET)
- SQL Server Database

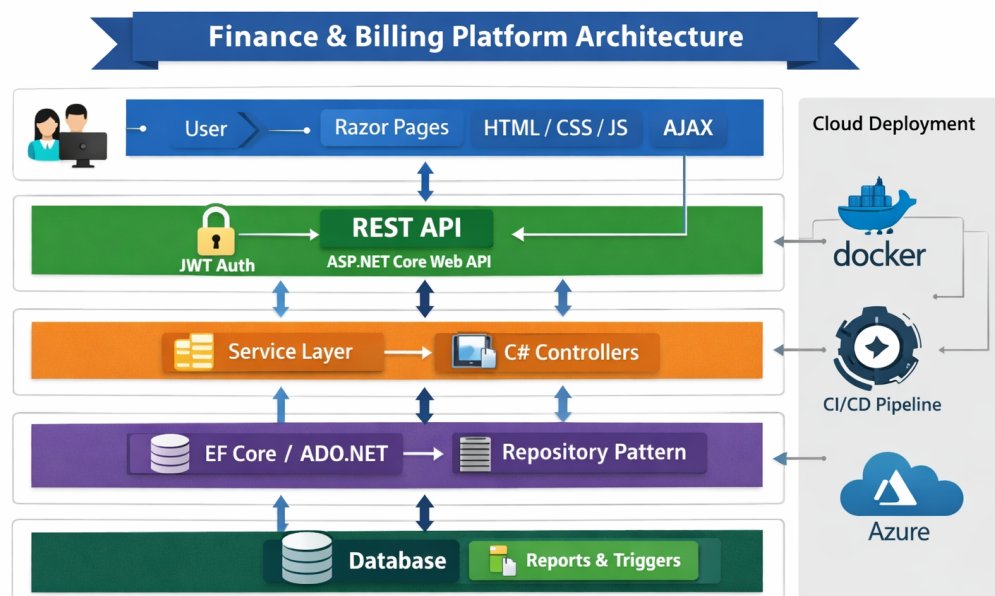


Figure 3.1: System Architecture Diagram – FinanceBillingPlatform

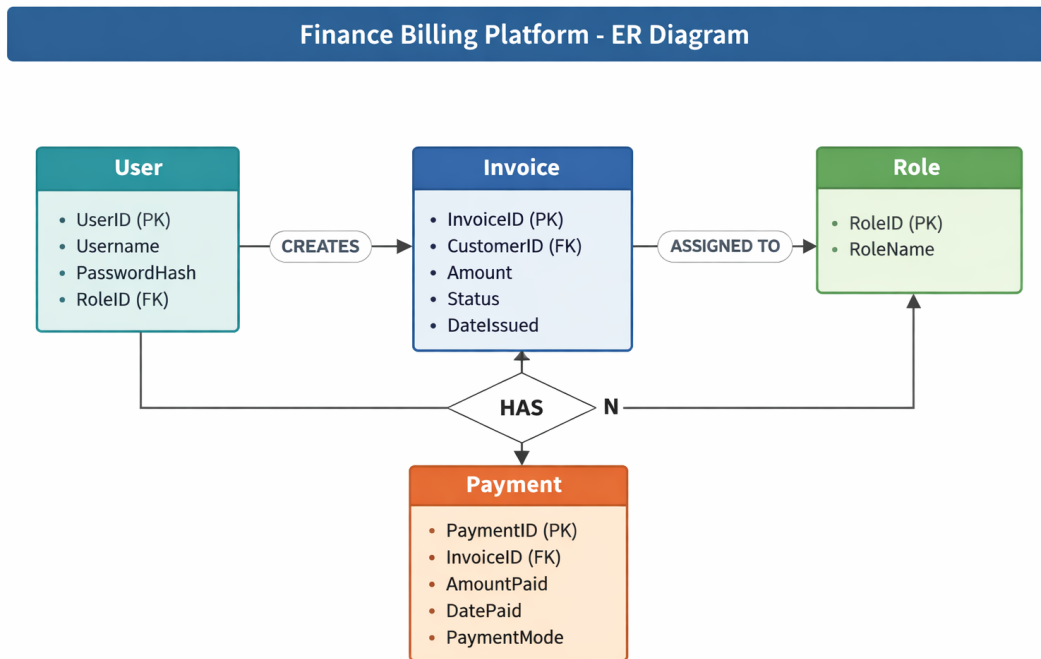


Figure 3.2: Entity Relationship Diagram – FinanceBillingPlatform

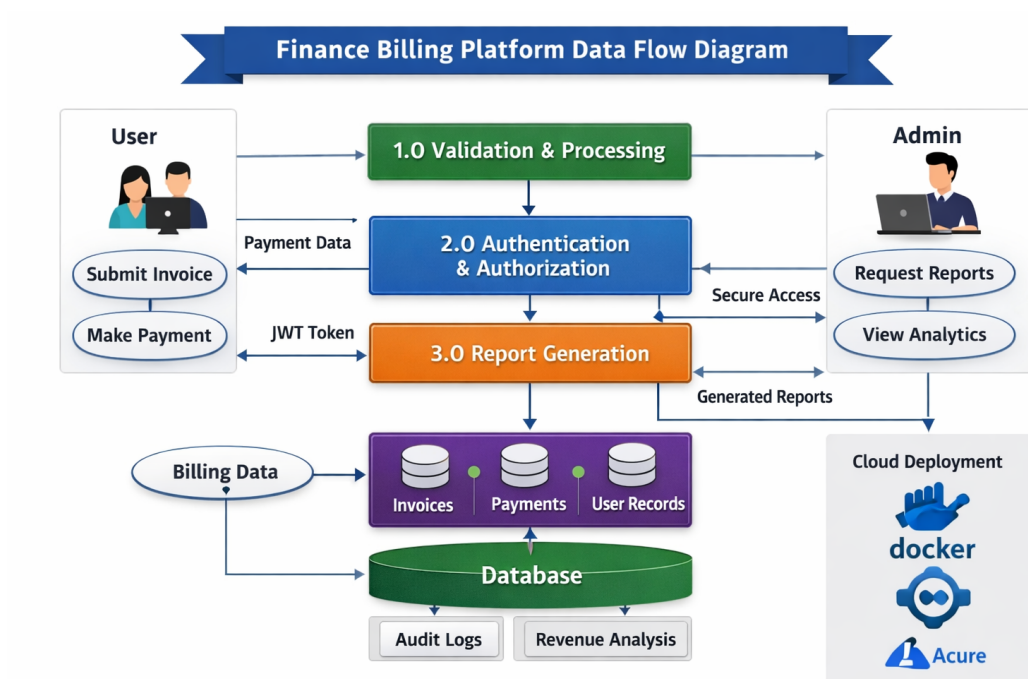


Figure 3.3: Data Flow Diagram – FinanceBillingPlatform

Chapter 4

Technology Stack

- Backend: .NET 8, ASP.NET Core, C# 12
- Database: SQL Server, EF Core
- Security: JWT, BCrypt password hashing
- Frontend: Razor Pages, HTML, CSS, AJAX
- DevOps: GitHub Actions, Docker, Azure App Services
- Testing: xUnit, Moq

Chapter 5

Project Structure

```
FinanceBillingPlatform/  
    FinanceBilling.Core/  
        Entities/  
            User.cs  
            Invoice.cs  
            Payment.cs  
            Role.cs  
        Interfaces/  
        DTOs/  
    FinanceBilling.Infrastructure/  
        Data/  
            FinanceBillingDbContext.cs  
        Repositories/  
        Services/  
        Security/  
    FinanceBilling.API/  
        Controllers/  
            AuthController.cs  
            InvoicesController.cs  
            PaymentsController.cs  
            UsersController.cs  
        Program.cs  
    Tests/  
    FinanceBillingPlatform.sln
```


Chapter 6

Core Components & Implementation

Entities

User.cs

```
public class User {  
    public int UserId { get; set; }  
    public string Username { get; set; }  
    public string Email { get; set; }  
    public string PasswordHash { get; set; }  
    public int RoleId { get; set; }  
    public Role Role { get; set; }  
    public ICollection<Invoice> Invoices { get; set; }  
}
```

Invoice.cs

```
public class Invoice {  
    public int InvoiceId { get; set; }  
    public int ClientUserId { get; set; }  
    public decimal TotalAmount { get; set; }  
    public DateTime DueDate { get; set; }  
    public InvoiceStatus Status { get; set; }  
    public ICollection<Payment> Payments { get; set; }  
}
```

Payment.cs

```
public class Payment {  
    public int PaymentId { get; set; }  
    public int InvoiceId { get; set; }  
    public decimal AmountPaid { get; set; }  
    public DateTime PaymentDate { get; set; }  
    public string PaymentMethod { get; set; }  
}
```

Services

AuthService.cs

```
public async Task<LoginResponseDto?> LoginAsync(LoginRequestDto
    dto) {
    var user = await _userRepository.GetByUsernameAsync(dto.
        Username);
    if (user == null || !BCrypt.Net.BCrypt.Verify(dto.Password,
        user.PasswordHash))
        return null;

    var token = _jwtTokenService.GenerateToken(user);
    return new LoginResponseDto { Token = token, Username = user.
        Username, Role = user.Role.RoleName };
}
```

InvoiceService.cs

```
public async Task CreateInvoiceAsync(CreateInvoiceDto dto) {
    var invoice = new Invoice {
        ClientUserId = dto.ClientUserId,
        TotalAmount = dto.TotalAmount,
        DueDate = dto.DueDate,
        Status = InvoiceStatus.Pending
    };
    await _invoiceRepository.AddAsync(invoice);
}
```

PaymentService.cs

```
public async Task AddPaymentAsync(CreatePaymentDto dto) {
    var payment = new Payment {
        InvoiceId = dto.InvoiceId,
        AmountPaid = dto.AmountPaid,
        PaymentDate = DateTime.UtcNow,
        PaymentMethod = dto.PaymentMethod
    };
    await _paymentRepository.AddAsync(payment);
    // Update invoice status logic
}
```

Chapter 7

API Endpoints

- POST /api/auth/register – Register new user
- POST /api/auth/login – JWT issuance
- POST /api/invoices – Create invoice
- GET /api/invoices – Get all invoices
- GET /api/invoices/client/{id} – Get client invoices
- POST /api/payments – Add payment
- GET /api/payments/invoice/{id} – Get payments for invoice
- GET /api/reports/revenue – Revenue leakage report

Chapter 8

Database Schema

FinanceBillingDbContext.cs

```
public class FinanceBillingDbContext : DbContext {  
    public DbSet<User> Users { get; set; }  
    public DbSet<Role> Roles { get; set; }  
    public DbSet<Invoice> Invoices { get; set; }  
    public DbSet<Payment> Payments { get; set; }  
    public DbSet<AuditLog> AuditLogs { get; set; }  
}
```

Chapter 9

Authentication & Security

JWT Configuration

```
"Jwt": {  
  "Key": "FinanceBillingSecretKey2025@123456789",  
  "Issuer": "FinanceBilling.API",  
  "Audience": "FinanceBilling.Client",  
  "ExpirationMinutes": 60  
}
```

Password Hashing

```
public string HashPassword(string password) {  
    return BCrypt.Net.BCrypt.HashPassword(password, 11);  
}
```

Chapter 10

Configuration

appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost\\SQLEXPRESS;Database=
      FinanceBillingDb;Trusted_Connection=True;"
  },
  "Jwt": {
    "Key": "FinanceBillingSecretKey2025@123456789",
    "Issuer": "FinanceBilling.API",
    "Audience": "FinanceBilling.Client",
    "ExpirationMinutes": 60
  }
}
```

Chapter 11

Getting Started

Prerequisites

- .NET 8 SDK installed
- SQL Server (SQLEXPRESS recommended)
- Visual Studio 2022 or Rider IDE
- Git for version control

Installation Steps

Step 1: Clone Repository

```
git clone https://github.com/ganeshvardhini/
  FinanceBillingPlatform.git
cd FinanceBillingPlatform
```

Step 2: Database Setup

```
# Create database
sqlcmd -S localhost\SQLEXPRESS -Q "CREATE DATABASE
  FinanceBillingDb"

# Apply migrations
cd FinanceBilling.API
dotnet ef database update --project ../FinanceBilling.
  Infrastructure
```

Step 3: Restore and Build

```
dotnet restore
dotnet build
```

Step 4: Run Application

```
cd FinanceBilling.API
dotnet run

# Application runs on: https://localhost:7000
# Swagger UI: https://localhost:7000/swagger
```

First Steps with API

1. Register as New User

```
curl -X POST https://localhost:7000/api/auth/register \
-H "Content-Type: application/json" \
-d '{
  "username": "testuser",
  "email": "test@example.com",
  "password": "TestPass@123"
}'
```

Response: "Registration submitted. Awaiting approval."

2. Login as Admin (Existing)

```
curl -X POST https://localhost:7000/api/auth/login \
-H "Content-Type: application/json" \
-d '{
  "username": "admin",
  "password": "Admin@123"
}'
```

Response:

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... ",
  "username": "admin",
  "role": "Admin"
}
```

3. Approve Pending User

```
curl -X POST https://localhost:7000/api/users/approve \
-H "Authorization: Bearer {ADMIN_TOKEN}" \
-H "Content-Type: application/json" \
-d '{
  "userId": 5,
  "roleId": 3
}'
```


4. Login as Approved User

```
curl -X POST https://localhost:7000/api/auth/login \
-H "Content-Type: application/json" \
-d '{
  "username": "testuser",
  "password": "TestPass@123"
}'
```

Response:

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "username": "testuser",
  "role": "Client"
}
```

Summary

At this point, the newly registered user has been approved by the Admin, can log in successfully, and receives a JWT token. This token must be included in the **Authorization** header for all subsequent API requests.

Example:

```
curl -X GET https://localhost:7000/api/invoices \
-H "Authorization: Bearer {USER_TOKEN}"
```

This ensures secure access to invoices, payments, and reports based on the user's assigned role.

Chapter 12

Testing

The FinanceBillingPlatform project uses **xUnit** and **Moq** for unit testing.

Test Coverage

Test Class	Methods	Coverage
AuthServiceTests	8	Register, Login, Password validation
UserServiceTests	5	Approvals, retrieval, client lookup
InvoiceServiceTests	4	Invoice creation, retrieval
PaymentServiceTests	6	Payment creation, status updates

Running Tests

```
# Run all tests
dotnet test

# Run specific test file
dotnet test Tests/AuthServiceTests.cs

# Run with verbose output
dotnet test --verbosity detailed

# Run with coverage
dotnet test /p:CollectCoverage=true
```

Chapter 13

Future Scope & Comparison

Future Enhancements

- AI-powered fraud detection
- Payment gateway integration (Stripe/PayPal)
- Dashboard analytics with charts
- Microservices architecture for scalability
- Multi-tenant SaaS support

Comparison with Live Applications

Feature	FinanceBillingPlatform	QuickBooks/Zoho
Tech Stack	ASP.NET Core, SQL Server	Proprietary
Security	JWT, Role-based	OAuth
Deployment	Docker + Azure	Cloud-native
Focus	Learning & analytics	Commercial finance